PCIe TLP Investigation Report

GSoC 2026 — Project 2.3: PCIe TLP Communication Framework using P4
P4 Language Consortium

Eslam Ashraf Ahmed
Electronics and Communications Engineering — Benha University
March 2026

Chapter 1: What is PCIe

A high-speed interface called Peripheral Component Interconnect Express facilitates communication between peripheral devices such as the GPU, SSD, FPGA accelerator, etc., with the CPU in a serial fashion. The PCIe interface facilitates serial communication between the peripheral devices, unlike the other interfaces that facilitate parallel communication. Two differential pairs are present in each lane of the PCIe interface: transmit and receive.

In this project, the peripheral device to be considered is the AMD Xilinx Alveo U50 FPGA accelerator card. The FPGA accelerator card is connected to the PCIe slot present on the server board. The PCIe interface is the only means by which the CPU communicates with the Alveo FPGA.

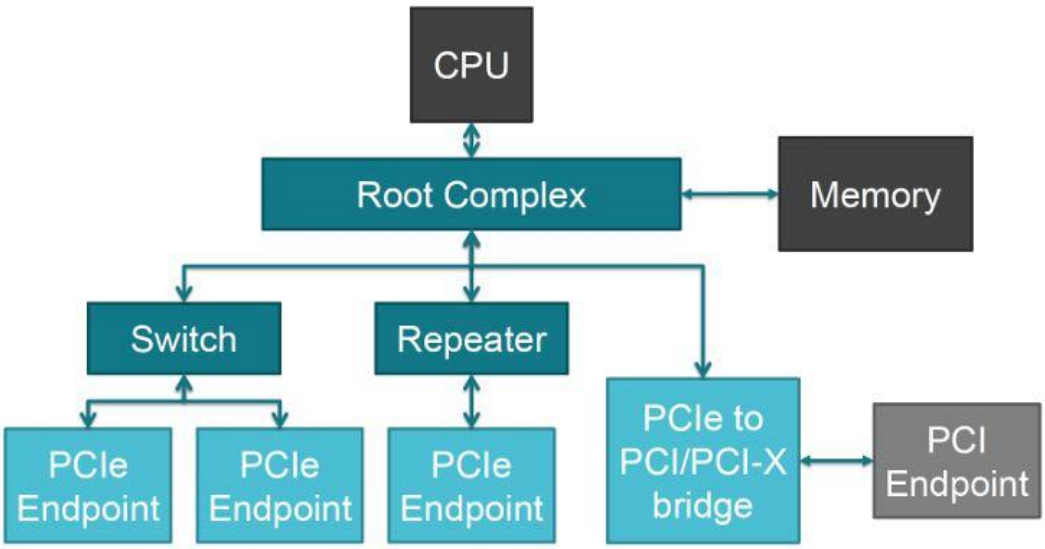

Figure 1 — PCIe Topology

Chapter 2: Transaction Layer Packets (TLPs)

TLP (Transaction Layer Packet) is the basic unit of data exchange in PCIe — it is the currency of communication between the host and the FPGA. Every single transaction happens through a TLP. Each TLP consists of two main parts: a header which contains the address, type, and size of the transaction, and an optional payload which carries the actual data.

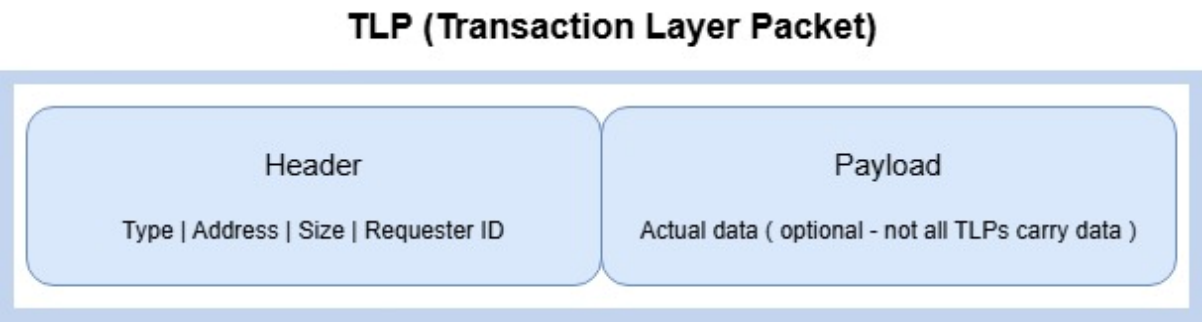

Figure 2 — TLP packet structure showing Header (always present) and Payload (optional)

TLP Types

There are four types of TLPs used in standard PCIe communication:

TLP Type	Sent By	Has Payload	Description
Memory Write Request	Host or FPGA	Yes	Sent when the host or FPGA wants to write data to the other side
Memory Read Request	Host or FPGA	No	Sent when the host or FPGA wants to read data from the other side
Completion with Data	Host or FPGA	Yes	Sent as a reply to a Memory Read Request, carrying the requested data
Completion without Data	Host or FPGA	No	Sent to confirm that a request was received and completed, with no data to return

Chapter 3: RQ/CQ/RC/CC Interfaces

The AMD Xilinx PCIe UltraScale+ IP exposes four AXI4-Stream interfaces to the FPGA user logic, each handling a different direction of TLP communication. All interfaces are described from the FPGA's perspective:

Interface	Full Name (PG213)	Direction	Description
RQ	Requester reQuest	FPGA → Host	Used when the FPGA sends a request (Memory Read/Write) to the host
CQ	Completer reQuest	Host → FPGA	Used when the host sends a request (Memory Read/Write) to the FPGA
RC	Requester Completion	Host → FPGA	Used when the host sends a completion reply to the FPGA's earlier request
CC	Completer Completion	FPGA → Host	Used when the FPGA sends a completion reply to the host's earlier request

Chapter 4: AXI4-Stream Interface

The Xilinx PCIe UltraScale+ IP uses AXI4-Stream protocol to transfer TLP packets between the PCIe core and the FPGA user logic. Each of the four interfaces (RQ/CQ/RC/CC) uses AXI Stream to carry TLP data beat by beat.

Data transfer occurs only when both TVALID and TREADY are asserted in the same clock cycle. If the user logic deasserts TREADY while TVALID is high, the core freezes and holds the data stable on the bus until TREADY is asserted again. TLAST marks the final beat of each TLP packet, while TUSER carries sideband descriptor information about the TLP being transferred.

Key AXI Stream Signals

Signal	Description
TDATA	Carries the actual TLP data across the interface
TVALID	Asserted by the core when data on TDATA is valid and ready to transfer
TREADY	Asserted by user logic when it is ready to accept data

Signal	Description
TLAST	Asserted on the last beat of a TLP packet to indicate end of packet
TKEEP	Indicates which Dwords in TDATA contain valid data (1 bit per Dword)
TUSER	Carries sideband descriptor and metadata about the TLP being transferred (161 bits on RC interface)

References

1. AMD Xilinx, PCIe UltraScale+ IP Product Guide (PG213), https://docs.amd.com/r/en-US/pg213-pcie4-ultrascale-plus

2. iHalt10, PCIe Subsystem for AMD Xilinx, https://github.com/iHalt10/pcie_subsystem

3. iHalt10, VNP4 Framework, https://github.com/iHalt10/vnp4_framework

4. P4 Language Consortium, GSoC 2026 Project 2.3, https://github.com/p4lang/gsoc/blob/main/2026/ideas_list.md